

# AEGIS — Setup Checklist

Personal environment setup tasks for building the MVP/POC. Complete these in order — each section unblocks the next.

**GCP Production Status (2026-02-24):** `aegis-prod-488120` is live. API: `https://api.aegisimaging.ai` — all services healthy, cloud smoke suite 11/11 PASS. DIMSE receiver: `aegis-prod-dimse-receiver` (Compute Engine VM, `us-central1-a`, static IP `35.232.172.221`, port 11112). CI/CD: Cloud Build triggers active in `us-central1` ( `deploy-on-develop` + `terraform-apply-on-develop` ).

**AWS Production Status (2026-02-25):** `301691475234 / us-east-1` is live. API: `https://aws.api.aegisimaging.ai` — 10 ECS Fargate services, RDS PostgreSQL 15, S3, ALB + Cognito auth. Cross-cloud DICOM routing: GCP→AWS STOW-RS verified live. GitHub Actions CI/CD active (fires on push to `develop` ).

**Azure Status (2026-02-26):** Deploying — day 9. Azure Container Apps + PostgreSQL Flexible Server + Azure Blob Storage + Azure Container Registry. GitHub Actions OIDC CI/CD ( `.github/workflows/deploy-azure.yml` ) fires on push to `develop` .

## Monitoring & Observability

**Cloud Monitoring Dashboard** (requires GCP console access):  
<https://console.cloud.google.com/monitoring/dashboards?project=aegis-prod-488120>

- Alert policies** (9 active):
- API 5xx rate, API p99 latency, API uptime check
  - Cloud SQL CPU, disk, connections
  - Cloud Run memory
  - Study stuck SLA, pipeline failures

## 0. Business & Account Setup

Each LLC gets its own accounts. Do not share accounts across AEGIS Imaging LLC and Encore Music LLC — keeps billing, sender reputation, and access controls separate per legal entity.

### Per-LLC accounts to create

| Service               | AEGIS Imaging LLC                       | Why separate                                                                |
|-----------------------|-----------------------------------------|-----------------------------------------------------------------------------|
| GCP                   | New project under AEGIS billing account | Billing tied to LLC, Terraform state isolated                               |
| Vercel                | New account (free Hobby tier)           | Separate deployment, billing, env vars                                      |
| Brevo                 | New account (free: 300 emails/day)      | Separate sender domain/reputation (@aegisimaging.ai VS @encoremusicapp.com) |
| Business bank account | Open for AEGIS Imaging LLC              | Required for GCP billing, separates finances                                |

**GitHub — use Organizations (one personal account is fine)**

- ☐ Create GitHub Organization `aegis-imaging` (free tier)
- ☒ Repo transferred to `aegis-imaging/aegis` — transfer complete

**P.O. Box (shared across all LLCs)**

- ☐ Get a P.O. Box (one box, shared by AEGIS Imaging LLC, Encore Music LLC, Matt Senjem Music LLC)
- ☐ Amend each LLC filing with MN Secretary of State (~\$35 each) to replace home address with P.O. Box

**Prerequisites checklist**

- ☐ AEGIS Imaging LLC EIN obtained ✓
- ☐ Open business bank account for AEGIS Imaging LLC
- ☒ Create GCP account + billing account for AEGIS Imaging LLC — project `aegis-prod-488120` , billing `016DEE-91CE5C-ECB970`
- ☐ Create Vercel account for AEGIS Imaging LLC
- ☐ Create Brevo account for AEGIS Imaging LLC (free tier: 300 emails/day)

---

**1. Local Development Environment**

- ☐ Install Go 1.24+ ( `brew install go` )
- ☐ Install Terraform 1.5+ ( `brew install terraform` )
- ☐ Verify Node.js 20+ and npm are installed ( `node --version` )
- ☐ Install Docker Desktop (for building/testing containers locally)
- ☐ Clone the AEGIS repo and verify the monorepo structure

**2. GCP Project Setup**

- ☒ Create a new GCP project under the **AEGIS Imaging LLC billing account** — `aegis-prod-488120` (region `us-central1` )
- ☒ Link the AEGIS billing account to the project — `016DEE-91CE5C-ECB970`
- ☒ Install the gcloud CLI ( `brew install google-cloud-sdk` )
- ☒ Authenticate: `gcloud auth login` and `gcloud auth application-default login`
- ☒ Set default project: `gcloud config set project aegis-prod-488120`

**Beta launch auth (GCP IAP)**

For the beta/MVP, use GCP Identity-Aware Proxy (IAP) to gate the admin dashboard. No custom auth flow needed — Google handles login.

| User type                      | Auth    | How                                      |
|--------------------------------|---------|------------------------------------------|
| Upload portal (external sites) | None    | Public — anyone with the link can upload |
| Admin dashboard (beta testers) | GCP IAP | Sign in with Google account              |

- ☒ Deploy Go API to Cloud Run — `https://api.aegisimaging.ai`

- ☒ Enable IAP on the Cloud Run load balancer
- ☒ **Provision the IAP service agent (one-time per project — run in Cloud Shell as project owner):**

```
gcloud beta services identity create \
  --service=iap.googleapis.com \
  --project=YOUR_PROJECT_ID
```

This creates `service-{PROJECT_NUMBER}@gcp-sa-iap.iam.gserviceaccount.com`. Terraform grants it `roles/run.invoker` on the admin Cloud Run service automatically — but the identity must exist first. Safe to run multiple times (idempotent). Run this **before** terraform apply in section 4.

- ☒ Add beta testers' Google accounts to IAP access list: ( `matthewsenjem@gmail.com` )

```
gcloud iap web add-iam-policy-binding \
  --member="user:tester@gmail.com" \
  --role="roles/iap.httpsResourceAccessUser"
```

- ☒ Add the same emails to `admin_users` table (role: `admin` or `viewer`) — `matthewsenjem@gmail.com` seeded via `FIRST_ADMIN_EMAIL`
- ☒ Set env vars on Cloud Run: `AUTH_ENABLED=true` `AUTH_PROVIDER=iap`
- ☒ Verify: tester visits admin dashboard URL → Google sign-in → dashboard loads

### 3. Terraform — Project Bootstrap

- ☒ Copy `terraform/project/terraform.tfvars.example` to `terraform/project/terraform.tfvars`
- ☒ Fill in your `project_id`, `region`, and `billing_account`
- ☒ Run `terraform init` in `terraform/project/`
- ☒ Run `terraform plan` and review the output
- ☒ Run `terraform apply` to enable all required GCP APIs
- ☒ Verify APIs are enabled: `gcloud services list --enabled`

### 4. Terraform — Infrastructure

- ☒ Copy `terraform/infra/terraform.tfvars.example` to `terraform/infra/terraform.tfvars`
- ☒ Fill in required `terraform/infra/terraform.tfvars` values:
  - `project_id = "aegis-prod-488120"`, `region = "us-central1"`, `environment = "prod"`
  - `api_domain = "api.aegisimaging.ai"`, `admin_domain = "admin.aegisimaging.ai"`
  - `iap_oauth_client_id`, `iap_oauth_client_secret`, `iap_access_members = ["user:matthewsenjem@gmail.com"]`
  - `db_password` (via Secret Manager), `db_password_secret_id`
  - image URLs for all services at `us-central1-docker.pkg.dev/aegis-prod-488120/aegis-services`

- ☒ Build and push images to Artifact Registry — all 8 services pushed at `:latest`
- ☒ Run `terraform init` in `terraform/infra/`
- ☒ Run `terraform fmt -check`
- ☒ Run `terraform validate`
- ☒ Run `terraform plan` and review
- ☒ Run `terraform apply` — created:
  - VPC/subnet/private-service networking/Cloud NAT
  - Artifact Registry repository ( `aegis-services` )
  - Cloud SQL PostgreSQL (private IP), Healthcare API dataset + DICOM stores, GCS buckets, Pub/Sub, BigQuery
  - Cloud Run services (API + 6 sidecars + admin dashboard)
  - Global HTTPS load balancer + managed cert + Cloud Armor + IAP admin backend
- ☒ Verify Artifact Registry repository exists:

```
gcloud artifacts repositories list --location=us-central1
```

- ☒ Verify Cloud Run services are deployed:

```
gcloud run services list --region=us-central1
```

- ☒ Verify staging bucket exists: `gsutil ls`
- ☒ Verify API health through LB domain:

```
curl -f https://api.aegisimaging.ai/healthz # returns {"status":"ok",...}
```

- ☒ Verify each sidecar health endpoint — all 6 sidecars (defacing, phi-detection, qc-service, bids-service, classification-service, protocol-service) return `{"status":"healthy"}` via `/health`
- ☒ Verify admin dashboard is gated by IAP — `https://admin.aegisimaging.ai` requires Google sign-in
- ☐ Verify SMTP egress static IP (not yet configured — email not enabled):

```
terraform output smtp_egress_ip
```

- ☐ Allowlist that IP on your SMTP relay/service
- ☐ Verify monitoring baseline exists:

```
gcloud monitoring policies list --format='value(displayName)'
```

- ☐ Open the Cloud Monitoring dashboard in GCP Console:

```
https://console.cloud.google.com/monitoring/dashboards?project=aegis-prod-488120
```

Expected: "AEGIS Operations — prod" dashboard with 11 tiles (API rate/errors/latency, Cloud Run instances/memory, Cloud SQL CPU/disk/connections, sidecar 5xx, pipeline failures, stuck-study SLA alerts).

- ☐ Confirm alert policies are active (9 total after Feature 55):

```
gcloud monitoring policies list --project=aegis-prod-488120 --
format='table(displayName,enabled)'
```

- ☐ Confirm log-based metrics exist:

```
gcloud logging metrics list --project=aegis-prod-488120 --
format='value(name)'
```

# Expected: aegis-prod-pipeline-failures, aegis-prod-study-stuck

- ☐ See `terraform/monitoring/README.md` for full metric reference and runbook links.
- ☐ See `docs/runbooks/alert-response.md` for per-alert incident response procedures (triage steps, remediation commands, escalation paths for all 9 alert policies).

## 4x. DIMSE Receiver — Compute Engine VM

The DIMSE C-STORE SCP runs on a dedicated Compute Engine VM because Cloud Run cannot expose raw TCP ports. Terraform creates the VM; Cloud Build deploys new images by updating VM metadata and resetting the instance.

- ☒ VM `aegis-prod-dimse-receiver` created in `us-central1-a` via `terraform/infra/dimse.tf`
- ☒ Static regional IP `35.232.172.221` assigned; firewall rule allows TCP 11112 from `0.0.0.0/0`
- ☒ VPC-internal firewall allows TCP 8080 from Go API → DIMSE VM health endpoint
- ☒ Startup script mounts GCS staging bucket via `gcsfuse` at `/app/data` and starts the DIMSE container
- ☒ `dimse_receiver_image` and `dimse_api_url` set in `terraform/infra/terraform.tfvars` (stored in Secret Manager as `aegis-prod-terraform-tfvars` version 2)
- ☐ Verify DIMSE receiver is running after a test C-STORE from PACS:

```
# From any host with storeescu installed
storeescu -v -aec AEGIS 35.232.172.221 11112 /path/to/test.dcm
# Then verify in admin dashboard → Studies tab
```

- ☐ Verify VM health endpoint (VPC-internal only):

```
# From a Cloud Run service or Cloud Shell with VPC access
curl http://<vm-internal-ip>:8080/healthz
```

**To update the DIMSE receiver image** (done automatically by Cloud Build on `develop` push):

```
# Manual update (if needed):
gcloud compute instances add-metadata aegis-prod-dimse-receiver \
  --zone=us-central1-a \
  --metadata=dimse-image=us-central1-docker.pkg.dev/aegis-prod-488120/aegis-
```

```
services/dimse-receiver:latest
gcloud compute instances reset aegis-prod-dimse-receiver --zone=us-central1-a
```

## 4y. Cloud Build CI/CD Triggers

Cloud Build triggers were created by `scripts/gcp_setup_cloudbuild.sh` and run in `us-central1`.

- ☒ GitHub App connection established (Cloud Build → `aegis` connection → `aegis-imaging/aegis` repo)
- ☒ Trigger `deploy-on-develop` — fires on push to `develop`; runs `cloudbuild.yaml` (builds+pushes all images, deploys Cloud Run services, hot-swaps DIMSE VM)
- ☒ Trigger `terraform-apply-on-develop` — fires when `terraform/infra/**` changes on `develop`; runs `cloudbuild.terraform.yaml`; reads `terraform.tfvars` from Secret Manager
- ☒ Cloud Build SA `aegis-cloud-build@aegis-prod-488120.iam.gserviceaccount.com` has all required IAM roles (tracked in `terraform/project/main.tf`)

**Monitor recent builds:**

```
gcloud builds list --project=aegis-prod-488120 --region=us-central1 --limit=5
```

**Re-run setup (idempotent):**

```
./scripts/gcp_setup_cloudbuild.sh
```

## 4a. First Admin Bootstrap

On the first deployment, the `admin_users` table is empty, so no one can log in to create admin users (chicken-and-egg). The API solves this with the `FIRST_ADMIN_EMAIL` env var — set it in `terraform/infra/terraform.tfvars` before `terraform apply`:

```
first_admin_email = "ops@aegisimaging.ai" # same as iap_access_members
```

Terraform sets `FIRST_ADMIN_EMAIL` on the API Cloud Run service. On startup, if `admin_users` is empty, the API seeds this email as the first admin (role: `admin`, enabled: `true`). Subsequent restarts are no-ops once any admin exists.

- ☒ Set `first_admin_email` in `terraform/infra/terraform.tfvars` before the first `terraform apply`
- ☒ Use the **same email address** as your first entry in `iap_access_members` so the user can log in immediately
- ☒ After apply, verify the admin was seeded by checking the API startup logs:

```
gcloud logging read 'resource.type="cloud_run_revision" AND
textPayload:"first-admin bootstrap: created admin user"' \
--project=YOUR_PROJECT_ID --limit=5 --format='value(textPayload)'
```

- ☒ Open the admin dashboard — `matthewsenjem@gmail.com` logs in, sees dashboard without "403 user not registered" error

To add more admins after the first login: use the **Users** tab in the admin dashboard, or call the API directly:

```
curl -X POST https://<api_domain>/api/admin-users \
-H "Authorization: Bearer <IAP_TOKEN>" \
-H "Content-Type: application/json" \
-d
'{"email":"colleague@example.com","name":"Alice","role":"admin","enabled":true}'
```

## 4b. Secrets Bootstrap and Rotation

### GCP (Cloud SQL + Cloud Run API)

- ☒ Confirm DB password secret exists — `aegis-prod-db-password` in Secret Manager ( `aegis-prod-488120` )
- ☒ Confirm API service account has secret accessor — verified via Terraform IAM binding
- ☐ Rotate DB password (dev drill — not yet done):

```
export NEW_DB_PASSWORD='<new-strong-password>'
gcloud secrets versions add aegis-dev-db-password --data-file==
<<<"$NEW_DB_PASSWORD"
gcloud sql users set-password aegis-api --instance=aegis-dev-postgres --
password="$NEW_DB_PASSWORD"
gcloud run services update aegis-api --region=us-central1 --update-env-
vars=ROTATION_EPOCH=$(date +%s)
```

- ☐ Verify post-rotation health:

```
curl -f https://<api_domain>/healthz
```

### AWS (RDS managed master credentials)

- ☐ Confirm RDS is configured with managed master password in AWS Secrets Manager:

```
aws rds describe-db-instances --db-instance-identifier aegis-postgres \
--query 'DBInstances[0].MasterUserSecret.SecretArn' --output text
```

- ☐ Rotate AWS RDS master credentials (dev drill):

```
aws rds modify-db-instance --db-instance-identifier aegis-postgres \
--rotate-master-user-password --apply-immediately
```

## 4c. Automated Cloud Smoke Suite

- ☒ Run the cloud smoke suite against deployed API:

```
SSL_CERT_FILE=/etc/ssl/cert.pem python3 scripts/cloud_smoke_test.py \
--base-url https://api.aegisimaging.ai \
--iap-email matthewsenjem@gmail.com
```

- ☒ Verify suite exits with status code `0` and prints all PASS steps — **11/11 PASS in 2.56s** (2026-02-22):
  - `healthz` ✓ `auth.me` ✓ `admin.users.registered` ✓
  - `upload.init` ✓ `upload.file` ✓ `upload.complete` ✓
  - `pipeline.progression` ✓ `study.approve` ✓
  - `share.create` ✓ `share.redeem` ✓ `share.download` ✓
- ☐ Verify fail-fast behavior:
  - re-run with an invalid admin header and confirm the suite fails quickly at `auth.me`
  - re-run with an invalid `--base-url` and confirm early transport failure

Manual trigger via GitHub Actions:

- Workflow: **Cloud Smoke** ( `.github/workflows/cloud-smoke.yml` )
- Set input `base_url` and (optional) repository secret `CLOUD_SMOKE_ADMIN_HEADER`

## 4d. Terraform — AWS HTTPS + Cognito Edge/Auth

**AWS Production Status (2026-02-25): LIVE.** `https://aws.api.aegisimaging.ai` — 10 ECS Fargate services, RDS PostgreSQL 15, S3, ALB + Cognito. Cross-cloud DICOM routing verified. GitHub Actions auto-deploy active.

- ☒ Copy `terraform/aws/terraform.tfvars.example` to `terraform/aws/terraform.tfvars`
- ☐ Fill required values:
  - `aws_region` , `environment` , `project_name`
  - `acm_certificate_arn` (issued cert in the same region as ALB)
  - `cognito_domain_prefix` (region-unique)
  - optional callback/logout URL overrides
- ☐ Run:

```
terraform -chdir=terraform/aws init
terraform -chdir=terraform/aws fmt -check
terraform -chdir=terraform/aws validate
terraform -chdir=terraform/aws plan
terraform -chdir=terraform/aws apply
```

- ☐ Verify HTTP to HTTPS redirect:

```
curl -I http://<alb_dns>
```

- Expect `301` redirect to `https://...`
- ☐ Verify unauthenticated protected path triggers Cognito auth:
  - Open `https://<alb_dns>/api/studies` in an incognito browser
  - Expect redirect/challenge to Cognito hosted UI

- ☐ Verify public path bypass remains available (for system health):

```
curl -f https://<alb_dns>/healthz
```

## 4e. Terraform — Azure Infrastructure

**Azure Status (2026-02-26): Deploying — Day 9.** Azure Container Apps + PostgreSQL Flexible Server + Azure Blob Storage + Azure Container Registry. GitHub Actions OIDC CI/CD ( `.github/workflows/deploy-azure.yml` ) auto-deploys on push to `develop` .

### Prerequisites

- ☐ Install Azure CLI: `brew install azure-cli` (macOS)
- ☐ Log in: `az login`
- ☐ Set subscription: `az account set --subscription <SUBSCRIPTION_ID>`
- ☐ Install Terraform: `brew install terraform`

### One-time: Create Terraform state storage

```
az group create --name aegis-tfstate --location eastus
az storage account create \
  --name aegistfstate \
  --resource-group aegis-tfstate \
  --sku Standard_LRS \
  --allow-blob-public-access false
az storage container create \
  --name tfstate \
  --account-name aegistfstate
```

### One-time: Create OIDC federated service principal for GitHub Actions

```
# Create service principal
SP=$(az ad sp create-for-rbac --name aegis-github-actions \
  --role Contributor \
  --scopes /subscriptions/<SUBSCRIPTION_ID> \
  --output json)

echo "Client ID:      $(echo $SP | jq -r .appId)"
echo "Tenant ID:      $(az account show --query tenantId -o tsv)"
echo "Subscription:    <SUBSCRIPTION_ID>"

# Add federated credentials for the develop branch
APP_ID=$(echo $SP | jq -r .appId)
az ad app federated-credential create \
  --id $APP_ID \
  --parameters '{"name":"aegis-develop","issuer":"https://token.actions.githubusercontent.com","subject":"repo:aegis-imaging/aegis:ref:refs/heads/develop","audiences":["api://AzureADTokenExchange"]}'
```

Add the following as **GitHub Secrets** on the repository:

- `AZURE_CLIENT_ID` — service principal App ID
- `AZURE_TENANT_ID` — Azure AD tenant ID
- `AZURE_SUBSCRIPTION_ID` — subscription ID
- `AZURE_ACR_REGISTRY` — set after Terraform apply (see outputs)
- `AZURE_RESOURCE_GROUP` — e.g. `aegis-prod`

Add as **GitHub Variables**:

- `AZURE_WEASIS_URL` — Weasis container app URL (set after first deploy)
- `AZURE_API_URL` — API container app URL (set after first deploy)

## Apply Terraform

- ☐ Copy `terraform/azure/terraform.tfvars.example` to `terraform/azure/terraform.tfvars`
- ☐ Fill required values:
  - `azure_ad_tenant_id` ( `az account show --query tenantId -o tsv` )
  - `db_admin_password` (strong password, ≥ 16 chars)
  - `alert_email`
  - `first_admin_email`
  - `api_domain` and `admin_domain` (optional — leave empty to use default ACA hostnames)

- ☐ Run:

```
terraform -chdir=terraform/azure init
terraform -chdir=terraform/azure fmt -check
terraform -chdir=terraform/azure validate
terraform -chdir=terraform/azure plan
terraform -chdir=terraform/azure apply
```

- ☐ Note the outputs:

```
terraform -chdir=terraform/azure output
```

Key outputs: `acr_login_server` , `api_url` , `admin_dashboard_url` , `acs_smtp_host`

## First image push

```
# Build and push images manually for first deploy (before GitHub Actions is
configured)
ACR=$(terraform -chdir=terraform/azure output -raw acr_login_server)
az acr login --name $ACR

# Build all images
docker build --platform linux/amd64 -t $ACR/api:latest api/
docker build --platform linux/amd64 \
  --build-arg VITE_WEASIS_BASE_URL=<weasis_url> \
  --build-arg VITE_API_BASE_URL=<api_url> \
  -t $ACR/admin-dashboard:latest frontend/admin-dashboard/
```

```
# ... repeat for all 13 services
```

```
docker push $ACR/api:latest
```

```
# ... push all
```

## Email (Azure Communication Services)

- ☐ After `terraform apply`, note the ACS SMTP config from outputs:
  - Host: `smtp.azurecomm.net`, Port: `587`
  - Username format: `<EntraAppClientId>|<TenantId>|<AcsResourceName>`
  - Password: OAuth2 access token (short-lived; use the ACS connection string for simpler SMTP)
- ☐ Alternatively: get the ACS connection string from Azure Portal → Communication Services → Keys
  - Use connection string auth for simpler SMTP integration with standard relay tools
- ☐ Set SMTP env vars on the API Container App:

```
az containerapp update --name aegis-prod-api \
  --resource-group aegis-prod \
  --set-env-vars \
    SMTP_HOST=smtp.azurecomm.net \
    SMTP_PORT=587 \
    SMTP_FROM=noreply@aegisimaging.ai \
    SMTP_USERNAME="<EntraAppClientId>|<TenantId>|<AcsResourceName>" \
    SMTP_PASSWORD="<oauth-token-or-access-key>"
```

## Verify deployment

- ☐ API health check:

```
curl -f https://<api_fqdn>/healthz | jq
```

- ☐ Auth test (Easy Auth injects header automatically when accessing admin dashboard):
  - Open admin dashboard URL in browser → Azure AD login → redirects back
  - Your Azure AD email must be added to `admin_users` table (via `FIRST_ADMIN_EMAIL` var)
- ☐ Cross-cloud routing test: add Azure→GCP destination in admin dashboard, upload a study, verify STOW-RS forward

## 5. Sample DICOM Data for Local Testing

- ☐ Download sample brain MRI DICOM files for testing (options below):
  - TCIA (The Cancer Imaging Archive): <https://www.cancerimagingarchive.net/>
  - dicom.offis.de sample files: <https://dicom.offis.de/dcmtdk/>
  - NEMA WG-6 sample DICOM: <ftp://medical.nema.org/medical/dicom/DataSets/>
- ☐ Place sample files in a local test directory (not committed to git)
- ☐ Verify files open in a DICOM viewer (e.g., Horos, 3D Slicer) to confirm validity

## 6. Frontend — Upload Portal

- ☐ `cd frontend/upload-portal && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3000>
- ☐ Install DICOM libraries: `npm install dcmjs dicom-parser`
- ☐ Test parsing a sample DICOM file in the browser console

## 7. Frontend — Admin Dashboard

- ☐ `cd frontend/admin-dashboard && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3001>
- ☐ Click **View** on any study row → Weasis viewer iframe appears inline
- ☐ Click **Open in new tab** → viewer opens in a new browser tab
- ☐ Click **Audit Log** tab → shows event table (empty until actions are taken)
- ☐ For a defaced head study: click **Review defacing** → side-by-side Weasis panel (Before/After)
- ☐ Click **Routing** tab → Destinations and Rules sections load (empty state)
- ☐ Click **Institutions** tab → Institutions table loads (empty state)

### 7a. Routing Rules Engine

- ☐ Open admin dashboard → **Routing** tab
- ☐ Create a Destination (type: `dicomweb`, any URL)
- ☐ Create a Routing Rule (e.g. `modality=MRI` → `auto_approve`) → appears in priority-ordered table
- ☐ Upload an MRI study via Upload Portal → study status auto-set to `approved`
- ☐ Verify routing log: `GET http://localhost:8080/api/studies/{id}/routing-log`
- ☐ Disable/enable rule toggle works; delete cleans up

### 7b. Institution Management

- ☐ Open admin dashboard → **Institutions** tab
- ☐ Create an institution (type: `sender`)
- ☐ Click **Projects** button on the row → inline project-link panel expands
- ☐ Link institution to a project: enter project UUID from `GET /api/projects`, choose role
- ☐ Verify link appears in table; unlink removes it
- ☐ Edit institution details; disable/enable toggle works

### 7c. Anonymization Profiles

- ☐ Open admin dashboard → **Profiles** tab
- ☐ Create a profile: name `Research`, project `default`, retained tags `PatientAge`, `StudyDate` → appears in table
- ☐ Click **Set default** → badge "default" appears on the row
- ☐ Upload a DICOM via the Upload Portal → check the de-identified tag diff: `PatientAge` and `StudyDate` should show action `K` (kept)
- ☐ Clear the default; re-upload → those tags are stripped again (normal Basic Profile)

- ☐ Edit / delete the profile

## 7d. Email Digest Subscriptions

- ☐ Start API with Mailpit enabled ( `SMTP_HOST=localhost SMTP_PORT=1025 go run .` )
- ☐ Open admin dashboard → **Notifications** tab
- ☐ Create a subscription: email `test@example.com` , project `default` , frequency `weekly`
- ☐ In the DB, force a digest due: `UPDATE digest_subscriptions SET last_sent_at = now() - interval '8 days'`
- ☐ Restart the API → scheduler fires on startup → check Mailpit at <http://localhost:8025> for the digest email
- ☐ Verify subject: `AEgis Weekly Summary – default – <date range>` , body has study counts, no PHI/UIDs
- ☐ Delete the subscription from the Notifications tab

## 7e. Admin Users

- ☐ Open admin dashboard → **Users** tab
- ☐ Create a user: email `admin@example.com` , name `Test Admin` , role `admin` → appears in table
- ☐ Edit → change role to `viewer` , add notes → save
- ☐ Disable the user → row greys out; re-enable
- ☐ Delete with confirm prompt
- ☐ Verify `admin_user.created` , `admin_user.updated` , `admin_user.deleted` appear in Audit Log tab

## 7f. Project Settings

- ☐ Open admin dashboard → **Projects** tab
- ☐ Click **+ New project** → enter name only (slug auto-generated) → create → appears in table
- ☐ Click **Edit** on the row → change description → save → description updates
- ☐ Try editing slug → warning hint is shown
- ☐ Verify `project.created` and `project.updated` in Audit Log tab
- ☐ `GET http://localhost:8080/api/projects/{id}` returns the updated project

## 7g. Upload Portal Features

### Project selector

- ☐ Create a second project in admin dashboard → Projects tab → **+ New project** (e.g. name: "Research")
- ☐ Open upload portal at <http://localhost:3000>
- ☐ Verify project dropdown appears with both projects listed
- ☐ Select the new project → upload a study → verify study appears under that project in admin dashboard
- ☐ Delete the second project → reload upload portal → dropdown is hidden (single project auto-selected)

## Drag-and-drop polish

- ☐ Drag a folder of DICOMs onto the drop zone → parsing starts with file count and total size displayed
- ☐ Click **Cancel** during parsing → returns to file selection
- ☐ Re-select files → on the preview screen, verify file count and size shown in the blue info bar
- ☐ Verify the **Confirm** button shows file count and size (e.g. "Confirm anonymization & upload (42 files, 156 MB)")
- ☐ During upload, verify current filename appears below the progress bar
- ☐ Click **Cancel** during upload → returns to preview with error message "Upload cancelled."

## Email validation

- ☐ On the preview screen, type an invalid email (e.g. "foo") → tab away → red validation error appears
- ☐ Verify **Confirm** button is disabled while email is invalid
- ☐ Clear the email field → button re-enables (email is optional)
- ☐ Enter a valid email → validation error disappears, helper text shows "You'll receive an email when your study is approved or rejected"

## Multi-study upload

- ☐ Prepare a folder containing DICOM files from 2+ different studies (different StudyInstanceUIDs)
- ☐ Upload the folder → portal detects multiple studies and shows a yellow notice
- ☐ Each study has its own StudySummary card with series/image count
- ☐ Click **Confirm & upload N studies** → each study uploads as a separate session
- ☐ Upload progress shows "Study X of N" during multi-study upload
- ☐ Success screen lists all session IDs and study UUIDs
- ☐ Verify each study appears as a separate row in the admin dashboard

## Auto-retry

- ☐ (Optional) Throttle network in DevTools mid-upload → verify retries up to 3x before error

## 7h. Burned-in PHI Detection

- ☐ Install Tesseract OCR: `brew install tesseract` (macOS) or `apt-get install tesseract-ocr` (Linux)
- ☐ Start the PHI detection service:

```
cd phi-detection && pip install -r requirements.txt
uvicorn app.main:app --port 8082
```

- ☐ Verify health: `curl http://localhost:8082/healthz` → should show `{"status":"ok","backend":"tesseract"}`
- ☐ Start the Go API with PHI detection enabled:

```
cd api && PHI_DETECTION_SERVICE_URL=http://localhost:8082 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_phi_scan` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows PHI Scan badge: **pending**
- ☐ Click **Scan for PHI** → badge changes to **scanning** → then **clean** or **flagged**
- ☐ Check Audit Log tab → `phi_scan.triggered` and `phi_scan.complete` entries appear
- ☐ Verify admin can still Approve a flagged study (flag is informational, not blocking)

## 7i. Batch Import CLI

- ☐ Build the CLI: `cd api && go build -o aegis-import ./cmd/import`
- ☐ Dry run (absolute path required): `./aegis-import --dir /absolute/path/to/test-data/brain-mri --project default --dry-run`
- ☐ Verify dry run output shows study count, modality, body part for each study
- ☐ Full import: `./aegis-import --dir /absolute/path/to/test-data/brain-mri --project default`
- ☐ Verify studies appear in admin dashboard with `source=internal`
- ☐ Verify `import.batch` entries in Audit Log tab
- ☐ Re-run the same import → should report duplicate errors (StudyInstanceUID unique constraint)
- ☐ External provenance test: `source=external` requires canonical institution selector (`institution_id` or `institution_slug`)
- ☐ Deprecated field test: posting unknown fields (for example `institution_ae_title`) to `/api/import/batch` returns HTTP 400
- ☐ Test API endpoint:

```
curl -X POST http://localhost:8080/api/import/batch \
  -H "Content-Type: application/json" \
  -d '{"dir":"/absolute/path/to/test-data/brain-mri","project_slug":"default","dry_run":true}'
```

## 7j. QC Automation Service

- ☐ Start the QC service:

```
cd qc-service && pip install -r requirements.txt
uvicorn app.main:app --port 8083
```

- ☐ Verify health: `curl http://localhost:8083/healthz` → should show `{"status":"ok","backend":"basic"}`
- ☐ Start the Go API with QC enabled:

```
cd api && QC_SERVICE_URL=http://localhost:8083 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_qc_check` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows QC badge: **pending**
- ☐ Click **Run QC** → badge changes to **checking** → then **pass**, **warn**, or **fail**

- ☐ Check Audit Log tab → `qc_check.triggered` and `qc_check.complete` entries appear
- ☐ Verify admin can still Approve a study with warn/fail QC status (status is informational)

## 7k. NifTI/BIDS Conversion Service

- ☐ Install dcm2niix: `brew install dcm2niix` (macOS) or `apt-get install dcm2niix` (Linux)
- ☐ Start the BIDS service:

```
cd bids-service && pip install -r requirements.txt
uvicorn app.main:app --port 8084
```

- ☐ Verify health: `curl http://localhost:8084/healthz` → should show `{"status":"ok","backend":"dcm2niix"}`
- ☐ Start the Go API with BIDS service enabled:

```
cd api && BIDS_SERVICE_URL=http://localhost:8084 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_bids_conversion` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows BIDS badge: **pending**
- ☐ Click **Convert to BIDS** → badge changes to **converting** → then **complete**
- ☐ Click **Download BIDS** → browser downloads a zip archive
- ☐ Unzip and verify BIDS structure: `dataset_description.json`, `participants.tsv`, `sub-*/anat/*.nii.gz` + `*.json`
- ☐ Check Audit Log tab → `bids_conversion.triggered` and `bids_conversion.complete` entries appear

## 7l. Metadata Classification Service

- ☐ Start the classification service:

```
cd classification-service && pip install -r requirements.txt
uvicorn app.main:app --port 8085
```

- ☐ Verify health: `curl http://localhost:8085/healthz` → should show `{"status":"ok","backend":"heuristic"}`
- ☐ Start the Go API with classification service enabled:

```
cd api && CLASSIFICATION_SERVICE_URL=http://localhost:8085 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_classification` (any modality)
- ☐ Upload a study with missing modality → study row shows Classification badge: **pending**
- ☐ Click **Classify** → badge changes to **classifying** → then **classified**
- ☐ Verify study modality and body\_part updated from "—" to classified values

- ☐ Verify routing rules re-evaluated (e.g. a `require_defacing` rule for HEAD now fires)
- ☐ Check Audit Log tab → `classification.triggered` and `classification.complete` entries appear

## 7m. MRI Protocol Compliance Service

- ☐ Start the protocol service:

```
cd protocol-service && pip install -r requirements.txt
uvicorn app.main:app --port 8086
```

- ☐ Verify health: `curl http://localhost:8086/healthz` → should show `{"status":"ok","backend":"basic"}`
- ☐ Start the Go API with protocol service enabled:

```
cd api && PROTOCOL_SERVICE_URL=http://localhost:8086 go run .
```

- ☐ Open admin dashboard → **Protocol Templates** tab
- ☐ Create a template: name `ADNI4 T1w`, project `default`, manufacturer `SIEMENS`, sequence type `T1w_MPRAGE`, add rules (e.g. RepetitionTime target 2300, tolerance 5%, severity warning)
- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_protocol_check` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows Protocol badge: **pending**
- ☐ Click **Check Protocol** → badge changes to **checking** → then **compliant**, **minor\_deviations**, or **non\_compliant**
- ☐ Check Audit Log tab → `protocol_check.triggered` and `protocol_check.complete` entries appear with per-parameter findings
- ☐ Verify admin can still Approve a non-compliant study (status is informational)
- ☐ Edit/delete the template from the Protocol Templates tab

## 7n. Authentication Middleware

Auth is disabled by default ( `AUTH_ENABLED=false` ) — all admin endpoints auto-authenticate as `ai@aegisimaging.ai` .

### Dev mode (default)

- ☐ Start the API: `cd api && go run .`
- ☐ Verify auth identity: `curl -s http://localhost:8080/api/auth/me | jq .` → shows `ai@aegisimaging.ai`, role `admin`
- ☐ Verify public routes work without auth: `curl -s http://localhost:8080/healthz` → `ok`
- ☐ Verify admin routes work without auth headers: `curl -s http://localhost:8080/api/studies | jq .total`
- ☐ Check Audit Log tab → audit entries show `ai@aegisimaging.ai` as the actor (not `admin` )

### Production mode (GCP IAP)

- ☐ Start API with auth enabled:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=iap go run .
```

- ☐ Verify unauthenticated request is blocked:

```
curl -s http://localhost:8080/api/studies | jq .  
# → {"error":"missing GCP IAP authentication header"}
```

- ☐ Verify unknown user is rejected:

```
curl -s -H "X-Goog-Authenticated-User-Email:  
accounts.google.com:unknown@test.com" \  
http://localhost:8080/api/studies | jq .  
# → {"error":"user not registered: unknown@test.com"}
```

- ☐ Register a user and verify access:

```
# First disable auth temporarily to create user  
cd api && go run .  
curl -s -X POST http://localhost:8080/api/admin-users \  
-H "Content-Type: application/json" \  
-d '{"email":"admin@test.com","name":"Test Admin","role":"admin"}'  
# Restart with auth enabled  
AUTH_ENABLED=true AUTH_PROVIDER=iap go run .  
curl -s -H "X-Goog-Authenticated-User-Email:  
accounts.google.com:admin@test.com" \  
http://localhost:8080/api/studies | jq .total  
# → 0 (or study count)
```

## Production mode (Azure AD)

- ☐ Start API with Azure AD auth:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=azure go run .
```

- ☐ Verify Azure header auth works:

```
curl -s -H "X-MS-CLIENT-PRINCIPAL-NAME: admin@test.com" \  
http://localhost:8080/api/studies | jq .total
```

## Production mode (AWS ALB + Cognito)

- ☐ Start API with AWS ALB auth:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=aws go run .
```

- ☐ Verify AWS ALB header auth works (simulate ALB-injected JWT):

```
# Create a base64url-encoded JWT payload with email claim
PAYLOAD=$(echo -n '{"email":"admin@test.com"}' | base64 | tr '+/' '-_' | tr -d '=')
JWT="header.${PAYLOAD}.signature"
curl -s -H "X-Amzn-Idc-Data: ${JWT}" \
  http://localhost:8080/api/studies | jq .total
```

## AWS S3 Storage

- ☐ Start API with S3 storage (LocalStack for dev):

```
# Start LocalStack (S3-compatible)
docker run -p 4566:4566 localstack/localstack
# Create bucket
aws --endpoint-url=http://localhost:4566 s3 mb s3://aegis-dev
# Run API with S3 backend
cd api && STORAGE_MODE=s3 S3_BUCKET=aegis-dev S3_REGION=us-east-1
S3_ENDPOINT=http://localhost:4566 go run .
```

- ☐ Upload a study via the Upload Portal → verify files stored in S3 bucket
- ☐ View study in Weasis → verify DICOMweb proxy retrieves from S3
- ☐ Verify signed URLs work: upload + download flows complete without error

## Azure AD App Registration Setup (for production Azure deployments)

### 1. Register the application in Azure Portal:

- Go to Azure Portal → Microsoft Entra ID → App registrations → New registration
- Name: AEGIS Admin Dashboard
- Supported account types: "Accounts in this organizational directory only" (Single tenant)
- Redirect URI: `https://<your-app-url>/.auth/login/aad/callback`
- Click Register

### 2. Configure authentication:

- Go to the app registration → Authentication
- Add platform: Web
- Redirect URIs: `https://<your-app-url>/.auth/login/aad/callback`
- Check "ID tokens" under Implicit grant

### 3. API permissions:

- Microsoft Graph → User.Read (delegated) — to read user email
- Grant admin consent for the directory

### 4. Configure the hosting platform:

- Azure App Service:** Go to Settings → Authentication → Add identity provider → Microsoft → Select the app registration
- Azure Application Gateway + AAD:** Configure AAD authentication on the gateway; it injects `X-MS-CLIENT-PRINCIPAL-NAME`
- Self-hosted with MSAL.js:** Add MSAL.js to the frontend; backend validates JWT Bearer tokens (future enhancement)

### 5. Add authorized users:

- In the AEGIS admin dashboard Users tab, add each Azure AD user's email as an admin user
- Their Azure AD login email must match the `admin_users.email` field (case-insensitive)

## 6. Environment variables:

```
AUTH_ENABLED=true  
AUTH_PROVIDER=azure # or "auto" to support both IAP and Azure
```

## 7o. RBAC Enforcement (Viewer Role)

The viewer role is read-only — viewers can browse all data but cannot create, update, delete, or trigger processing.

### Backend verification

- ☐ Create a viewer user (with API running in default dev mode):

```
curl -s -X POST http://localhost:8080/api/admin-users \  
-H "Content-Type: application/json" \  
-d '{"email":"viewer@aegisimaging.ai","name":"Test  
Viewer","role":"viewer","enabled":true}'
```

- ☐ Restart API as viewer: `cd api && DEV_USER_EMAIL=viewer@aegisimaging.ai go run .`
- ☐ Verify read endpoints work:

```
curl -s http://localhost:8080/api/studies | jq .total # → 200 OK  
curl -s http://localhost:8080/api/institutions | jq . # → 200 OK  
curl -s http://localhost:8080/api/auth/me | jq .role # → "viewer"
```

- ☐ Verify write endpoints are blocked:

```
curl -s -X POST http://localhost:8080/api/projects \  
-H "Content-Type: application/json" \  
-d '{"name":"test"}' | jq .  
# → {"error":"insufficient permissions: requires admin role"}
```

### Frontend verification

- ☐ Open admin dashboard at <http://localhost:3001> as viewer
- ☐ Verify Users tab is NOT visible in navigation
- ☐ Verify Studies tab: View + Download BIDS visible; Approve/Reject/Share/processing buttons hidden
- ☐ Verify Routing tab: data visible; Add/Edit/Delete/Toggle buttons hidden
- ☐ Verify Institutions tab: data visible; Add/Edit/Delete/Toggle/Link/Unlink hidden; Projects view button visible
- ☐ Verify Profiles, Protocol Templates, Notifications, Projects tabs: data visible; create/edit/delete buttons hidden
- ☐ Switch back to admin: restart API with `DEV_USER_EMAIL=ai@aegisimaging.ai` (or default) — all buttons return

## 7p. Defacing Service Backends

The defacing service supports multiple pluggable backends selected via the `DEFACE_TOOL` env var. DeepDefacer (3D U-Net) is the recommended production default for speed.

### Local verification (without Docker)

- ☐ Install dependencies:

```
cd defacing
pip install -r requirements.txt
pip install deepdefacer
```

- ☐ Start with DeepDefacer:

```
DEFACE_TOOL=deepdefacer uvicorn app.main:app --port 8081
```

- ☐ Verify health endpoint:

```
curl http://localhost:8081/healthz | python3 -m json.tool
# → {"status":"ok","backend":"deepdefacer","available":true}
```

- ☐ Test explicit backend selection:
  - `DEFACE_TOOL=nibabel` → health shows `nibabel-fallback`
  - `DEFACE_TOOL=mri_deface` → falls back to `nibabel-fallback` (unless `mri_deface` installed)
  - `DEFACE_TOOL=auto` → uses first available (deepdefacer if installed)

### Docker verification

- ☐ Build with DeepDefacer:

```
docker compose build defacing
# docker-compose.yml sets INCLUDE_DEEPDEFACER=true by default
```

- ☐ Start defacing service:

```
docker compose up defacing
```

- ☐ Verify via API health: `curl http://localhost:8080/healthz | python3 -m json.tool` → `services.defacing` shows healthy

## 7q2. Cloud AI Backends (PHI Detection + Classification)

Both the PHI detection and classification services support pluggable cloud AI backends. Cloud backends are optional — the services work with local-only backends (Tesseract, heuristic) by default. Cloud backends offer better accuracy for edge cases (burned-in text in poor image quality, missing DICOM tags).

### GCP Cloud Vision setup

- ☐ Enable the Cloud Vision API in your GCP project:

```
gcloud services enable vision.googleapis.com
```

- ☐ Create a service account for AEGIS:

```
gcloud iam service-accounts create aegis-vision \
  --display-name="AEGIS Cloud Vision"
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:aegis-vision@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

- ☐ Download the key (local dev only — use workload identity in production):

```
mkdir -p secrets
gcloud iam service-accounts keys create secrets/gcp-key.json \
  --iam-account=aegis-vision@$PROJECT_ID.iam.gserviceaccount.com
```

- ☐ Set env var: `export GOOGLE_APPLICATION_CREDENTIALS=./secrets/gcp-key.json`
- ☐ Build Docker images with Cloud Vision:

```
INCLUDE_GOOGLE_VISION=true docker compose build phi-detection classification-
service
```

- ☐ Start and verify:

```
docker compose up -d phi-detection classification-service
curl -s http://localhost:8082/healthz | python3 -m json.tool # → backend:
google_vision
curl -s http://localhost:8085/healthz | python3 -m json.tool # → backend:
google_vision
```

## AWS Textract / Rekognition setup

- ☐ Create an IAM user or role with these policies:
  - PHI detection: `AmazonTextractFullAccess`
  - Classification: `AmazonRekognitionReadOnlyAccess`
- ☐ Set AWS credentials:

```
export AWS_ACCESS_KEY_ID=<your-key>
export AWS_SECRET_ACCESS_KEY=<your-secret>
export AWS_DEFAULT_REGION=us-east-1
```

- ☐ Build Docker images with AWS backends:

```
INCLUDE_AWS_TEXTTRACT=true INCLUDE_AWS_REKOGNITION=true docker compose build  
phi-detection classification-service
```

- ☐ Override backend selection (if you want AWS instead of auto):

```
PHI_T00L=aws_texttract CLASSIFY_T00L=aws_rekognition docker compose up -d phi-  
detection classification-service
```

## Verify cloud backend selection

- ☐ Check health endpoints after starting services:

```
curl -s http://localhost:8082/healthz | python3 -m json.tool # PHI detection  
curl -s http://localhost:8085/healthz | python3 -m json.tool #  
classification
```

- ☐ Backend field shows the active backend ( google\_vision , aws\_texttract ,  
aws\_rekognition , tesseract , or heuristic )
- ☐ Auto mode selects: PHI: google\_vision > aws\_texttract > tesseract. Classification: google\_vision  
> aws\_rekognition > heuristic.
- ☐ Upload a study and trigger PHI scan / classification → results should come from the cloud  
backend
- ☐ Test fallback: unset cloud credentials → restart → falls back to local backend

## 7r. Study Export & DICOM Download

### Admin DICOM Download

- ☐ Upload and approve a study
- ☐ In the admin dashboard, click **Download DICOM** on the approved study row
- ☐ Verify a zip file downloads containing all DICOM files ( .dcm )
- ☐ Verify unapproved studies do NOT show the Download DICOM button

### Export Portal (Share Recipient Download)

- ☐ Create an export share for an approved study (admin dashboard → Share panel)
- ☐ Copy the share token from the API response
- ☐ Open the export portal: `http://localhost:3004?token={token}`
- ☐ Verify study info is displayed: modality, body part, file count, description, expiry
- ☐ Click **Download All (ZIP)** → zip file downloads
- ☐ Test expired/revoked token → clean error message displayed
- ☐ Test invalid token → "Share not found" error

### Export Forwarding (DICOMweb STOW-RS)

- ☐ Open admin dashboard → **Routing** tab
- ☐ Create a Destination (type: dicomweb , URL of a DICOMweb endpoint)
- ☐ Create a Routing Rule: action `require_export` + `route_to` (select the destination)

- ☐ Upload a study → verify `export_required=true` and `export_status=pending` in the studies table
- ☐ Approve the study → verify export auto-dispatches (status changes to `exporting` → `exported` )
- ☐ Check Audit Log tab → `export.triggered` and `export.complete` entries appear
- ☐ Test manual re-trigger: click **Export** button on a failed export → retries forwarding

## Export Portal Dev Server

```
cd frontend/export-portal && npm install && npm run dev # runs on :3004, proxies /api to :8080
```

## 7q. Automated Processing Pipeline

The pipeline auto-dispatches processing services after upload. Enabled by default ( `PIPELINE_AUTO=true` ).

- ☐ Configure routing rules that require multiple services (e.g. `require_classification` + `require_defacing` + `require_qc_check` + `require_bids_conversion` )
- ☐ Upload a study via the upload portal
- ☐ Check API logs for `pipeline: dispatching classification for ...` — classification runs first
- ☐ After classification completes, verify logs show parallel dispatch of defacing + PHI scan (if configured)
- ☐ After defacing completes, verify QC check and BIDS conversion are auto-dispatched
- ☐ Verify audit log shows `pipeline.dispatch` entries for each service
- ☐ Test manual override: click a manual trigger button in the dashboard — should still work
- ☐ Test disable: set `PIPELINE_AUTO=false` , upload again — no auto-dispatch, manual buttons required

## 7r2. Study Detail Page

The admin dashboard now includes a study detail view. Clicking a study UID in the studies table opens a dedicated panel.

- ☐ Open admin dashboard: `http://localhost:3001`
- ☐ Click any study UID link in the studies table → detail panel opens
- ☐ Verify header shows full study UID, status badge, source badge, description
- ☐ Verify meta row shows modality, body part, files, series, store, timestamps
- ☐ Verify pipeline visualization shows 7 stages with color-coded dots
- ☐ Test action buttons: Approve, Reject, Classify, Scan for PHI, etc.
- ☐ Click "View" → Weasis viewer opens inline
- ☐ For approved studies: verify share form appears, create a share link
- ☐ Check Audit Trail tab → shows all audit entries for this study
- ☐ Check Routing Log tab → shows routing rule evaluations
- ☐ Check Shares tab → shows created export shares

- ☐ Click "← Back to studies" → returns to the studies table
- ☐ API endpoints: `GET /api/studies/{id}` and `GET /api/studies/{id}/audit`

## 7s. Automated Tests (Go API)

Three-tier test suite: unit tests (no Docker), model integration tests (real PostgreSQL via testcontainers), and handler HTTP tests (httptest + real DB + temp storage).

### Unit tests only (no Docker required)

- ☐ Run: `cd api && go test -short -v ./...`
- ☐ Verify ~55 tests pass: routing engine, auth JWT parsing, CORS, config, email templates, storage, slugify

### Full test suite (requires Docker)

- ☐ Ensure Docker Desktop is running
- ☐ Run: `cd api && go test -v -count=1 ./...`
- ☐ Verify ~120 tests pass including model CRUD and handler HTTP tests
- ☐ Run with race detector: `cd api && go test -race -count=1 ./...`

### Test helpers ( `api/testutil/` )

- `TestDB(t)` — spins up PostgreSQL 15 via testcontainers, runs migrations, auto-cleanup
- `TestServer(t, db)` — creates `handler.Server` with temp local storage, pipeline disabled
- `SeedProject(t, db) / CreateTestStudy(t, db, projectID) / CreateTestAdminUser(t, db, email, role)` — test fixtures

### Makefile targets

```
make test      # full suite (requires Docker)
make test-unit # unit tests only (no Docker)
make test-race # full suite with race detector
```

## 8. Go API

- ☐ `cd api && go run .` — verify health endpoint at <http://localhost:8080/healthz>
- ☐ Add required Go dependencies as needed (storage SDKs, auth providers)
- ☐ Implement signed URL generation endpoint
- ☐ Test upload flow: browser → signed URL → GCS staging bucket

## 8a. Local Services (Docker Compose)

`docker-compose.yml` starts the **full platform stack** — database, email, viewer, Go API, and all Python services:

- ☐ `docker compose up -d` — builds and starts all 11 services
- ☐ Verify API health: `curl http://localhost:8080/healthz | python3 -m json.tool`
  - Should show `"status":"ok"`, `"database":"healthy"`, `"storage":"healthy"`, and all configured sidecar services as `"healthy"`

- ☐ Verify Weasis loads at <http://localhost:3005>
- ☐ Verify Mailpit web UI at <http://localhost:8025>

Services started by `docker compose up` :

| Service                | Port                                  | Notes                                                  |
|------------------------|---------------------------------------|--------------------------------------------------------|
| postgres               | 5432                                  | Data in pgdata named volume (persists across restarts) |
| mailpit                | 1025 / 8025                           | SMTP capture + web UI                                  |
| weasis                 | 3005                                  | Weasis DWV viewer                                      |
| api                    | 8080                                  | Go API (runs migrations on startup)                    |
| defacing               | (internal)                            | Defacing service                                       |
| phi-detection          | (internal)                            | Burned-in PHI detection                                |
| qc-service             | (internal)                            | QC automation                                          |
| bids-service           | (internal)                            | NIfTI/BIDS conversion                                  |
| classification-service | (internal)                            | Metadata classification                                |
| protocol-service       | (internal)                            | MRI protocol compliance                                |
| dimse-receiver         | 11112 (DICOM), 8080 (internal health) | DIMSE C-STORE ingress adapter                          |

Start individual services:

```
docker compose up -d postgres    # just database
docker compose up -d api         # API + postgres (auto-dependency)
docker compose build             # rebuild all images after code changes
docker compose down -v          # stop + destroy volumes (fresh start)
```

**Note:** The frontends (upload-portal on :3000, admin-dashboard on :3001) still run via `npm run dev` outside Docker, proxying `/api` to `localhost:8080` .

## 8aa. DIMSE PACS E2E Validation Harness

- ☐ Install DIMSE receiver dependencies:

```
cd dimse-receiver && pip install -r requirements.txt -r requirements-test.txt
```

- ☐ Run harness:

```
python3 scripts/dimse_pacs_e2e_harness.py
```

- ☐ Verify all four scenarios pass:

- `success_c_store_ingest`
- `transient_failure_to_retry_queue`
- `process_controls_restore_ingestion`
- `dead_letter_path_and_recovery`
- ☐ Admin dashboard verification:
  - Open Admin Dashboard → **DIMSE Ops** tab
  - Verify summary cards show pending/dead-letter counters
  - Trigger one control action (for example, `Process due` ) and verify counters/actions refresh
- ☐ Alerting verification (optional but recommended):
  - Set `DIMSE_RETRY_ALERTS_ENABLED=true`
  - Configure at least one threshold ( `DIMSE_RETRY_ALERT_DEAD_LETTER_NONZERO=true` or age thresholds)
  - Trigger threshold condition and verify `GET /ingest/retry/alerts` returns alert entries
  - Verify Admin Dashboard **DIMSE Ops** tab shows **Recent Retry Alerts**
- ☐ Verify durable retry state (restart-safe):
  - Ensure `DIMSE_INGEST_DURABLE_STORE_ENABLED=true` (default in `docker-compose.yml` )
  - Confirm state file path is on shared volume ( `DIMSE_INGEST_DURABLE_STORE_PATH` , default `/app/data/dimse-ingest-retry-state.json` )
  - Create at least one pending/dead-letter entry, restart `dimse-receiver` , and verify `/ingest/retry` counters persist
- ☐ On failure, review printed `dimse-receiver` log tail and rerun with `--keep-logs`
- ☐ Save run output as pilot evidence

Runbook:

- `docs/planning/dimse-pacs-e2e-validation-runbook.md`
- `docs/dicom-conformance.md` — DICOM conformance statement (SOP classes, transfer syntaxes, DICOMweb services, DIMSE services, de-identification profile)

## 8ab. DICOM File Retention & Cleanup Policy

### File lifecycle

DICOM files are written to shared storage at:

```
dicom/raw/{studyInstanceUID}/{index}.dcm    ← original tag-de-identified files
dicom/clean/{studyInstanceUID}/{index}.dcm  ← defaced output (when defacing is
required)
```

The **DIMSE receiver** writes to `dicom/raw/` when it receives a C-STORE. The **upload portal** and **batch import CLI** also write to `dicom/raw/` . These files are the canonical storage copy — no automatic cleanup runs. Files accumulate until explicitly managed.

### When is it safe to delete?

Studies in a terminal state are safe to archive or delete:

| Status                      | Safe to delete raw files? | Notes                                                 |
|-----------------------------|---------------------------|-------------------------------------------------------|
| approved                    | Yes                       | Defaced copy in dicom/clean/ if defacing was required |
| rejected                    | Yes                       | No further processing will occur                      |
| defaced                     | Raw only                  | Keep dicom/clean/ until approved or rejected          |
| received / defacing / clean | No                        | Still in active pipeline                              |

Do **not** delete files for studies still in the processing pipeline — the sidecars read from shared storage and will fail with missing-file errors.

## Production cleanup (GCS)

Set a GCS object lifecycle rule on the DICOM bucket so raw files transition automatically:

```
# Create lifecycle config (adjust age to your retention policy)
cat > /tmp/lifecycle.json << 'EOF'
{
  "lifecycle": {
    "rule": [
      {
        "action": { "type": "SetStorageClass", "storageClass": "COLDLINE" },
        "condition": { "age": 90, "matchesPrefix": ["dicom/raw/"] }
      },
      {
        "action": { "type": "SetStorageClass", "storageClass": "COLDLINE" },
        "condition": { "age": 365, "matchesPrefix": ["dicom/clean/"] }
      }
    ]
  }
}
EOF
gsutil lifecycle set /tmp/lifecycle.json gs://YOUR_GCS_BUCKET
```

For hard deletion instead of Coldline transition, change `"type": "Delete"` and remove `"storageClass"`.

**Note:** GCS lifecycle rules apply only to object age, not study status. Coordinate retention periods with your data governance policy and any IRB or DUA requirements.

## Production cleanup (S3)

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket YOUR_S3_BUCKET \
  --lifecycle-configuration '{
    "Rules": [
      {
```

```

    "ID": "dicom-raw-coldline",
    "Filter": { "Prefix": "dicom/raw/" },
    "Status": "Enabled",
    "Transitions": [{ "Days": 90, "StorageClass": "GLACIER_IR" }]
  }
]
}'

```

## Local dev cleanup

```

# Remove all DICOM files for one study (safe once approved/rejected)
rm -rf ./data/dicom/raw/{studyUID}
rm -rf ./data/dicom/clean/{studyUID}

# Full reset (destroys all data including PostgreSQL)
docker compose down -v

# BIDS output cleanup
rm -rf ./data/bids/{studyUID}

```

## DIMSE retry state file

The durable retry state file ( `dimse-ingest-retry-state.json` , default path `/app/data/dimse-ingest-retry-state.json` ) is small and safe to leave in place. It is overwritten on each retry worker pass. Delete it only to reset the queue to empty (pending/dead-letter entries are lost):

```

rm /app/data/dimse-ingest-retry-state.json
# Then restart dimse-receiver - it starts with empty queues

```

## 8b. Email (Local Dev with Mailpit)

Email is disabled by default — all calls are silent no-ops when `SMTP_HOST` is unset. Mailpit is included in `docker-compose.yml` (step 8a). To run standalone:

Email is disabled by default — all calls are silent no-ops when `SMTP_HOST` is unset.

- ☐ Run Mailpit (or use `docker compose up -d mailpit`):

```
docker run -p 1025:1025 -p 8025:8025 axllent/mailpit
```

- ☐ Start the API with SMTP env vars:

```
cd api && SMTP_HOST=localhost SMTP_PORT=1025 go run .
```

- ☐ Open Mailpit web UI at <http://localhost:8025>
- ☐ Trigger each notification to verify:
  - Upload a study via the portal — enter your email in the "Your email (optional)" field on the preview screen

- Approve the study in the admin dashboard → uploader gets approval email
- Reject a study → uploader gets rejection email
- Create a share for an approved study → recipient gets share email with download link
- ☐ Verify no crash when `SMTP_HOST` is unset (email silently skipped, no error returned)
- ☐ Verify upload with no email entered still completes successfully (no notification sent)

## 8c. Landing Page & Contact Form (Vercel + Brevo)

**Site:** aegisimaging.ai | **Source:** `frontend/landing/` | **Framework:** Vite + React

The landing page is deployed to Vercel as a static site with a serverless function for the contact form. Vercel auto-deploys on every push to the connected branch — no manual deploys needed after initial setup. Every PR also gets a preview URL.

### Step 1: Set up Vercel project

- ☐ Create a Vercel account for AEGIS Imaging LLC (separate from Encore) at [vercel.com](https://vercel.com) (free Hobby plan)
- ☐ Click **Add New → Project**
- ☐ Connect your GitHub account and select the **AEGIS** repo
- ☐ **Configure Project:**

| Setting                 | Value                                |
|-------------------------|--------------------------------------|
| <b>Root Directory</b>   | <code>frontend/landing</code>        |
| <b>Framework Preset</b> | Vite (should auto-detect)            |
| <b>Build Command</b>    | <code>npm run build</code> (default) |
| <b>Output Directory</b> | <code>dist</code> (default)          |

- ☐ Click **Deploy** — wait for the first build to succeed
- ☐ Note the preview URL Vercel gives you (e.g. `aegis-abc123.vercel.app` )

### Step 2: Configure contact form email (Brevo SMTP)

The contact form uses a Vercel serverless function ( `api/contact.ts` ) that sends email via Brevo SMTP. Without these env vars, submissions are logged to the console but not emailed — you can configure this later.

- ☐ Create a Brevo account for AEGIS Imaging LLC at [brevo.com](https://brevo.com) (free tier: 300 emails/day)
- ☐ Go to **Settings → SMTP & API → SMTP** and note your credentials
- ☐ In your Vercel project: **Settings → Environment Variables**, add:

| Key                          | Value                             | Environments        |
|------------------------------|-----------------------------------|---------------------|
| <code>BREVO_SMTP_HOST</code> | <code>smtp-relay.brevo.com</code> | Production, Preview |
| <code>BREVO_SMTP_PORT</code> | <code>587</code>                  | Production, Preview |
| <code>BREVO_SMTP_USER</code> | <i>(your Brevo SMTP login)</i>    | Production, Preview |
| <code>BREVO_SMTP_PASS</code> | <i>(your Brevo SMTP password)</i> | Production, Preview |

|                 |                                 |                     |
|-----------------|---------------------------------|---------------------|
| BREVO_SMTP_FROM | AEGIS <noreply@aegisimaging.ai> | Production, Preview |
|-----------------|---------------------------------|---------------------|

- ☐ Redeploy (Settings → Deployments → click : on latest → **Redeploy**)
- ☐ Test: submit the contact form → check inbox at `contact@aegisimaging.ai`

### Step 3: Add custom domain in Vercel

- ☐ In your Vercel project: **Settings → Domains**
- ☐ Type `aegisimaging.ai` and click **Add**
- ☐ Vercel will show the DNS records you need — keep this page open

### Step 4: Configure DNS at GoDaddy

- ☐ Log in to [godaddy.com](https://godaddy.com)
- ☐ Go to **My Products → aegisimaging.ai → DNS → Manage DNS**
- ☐ Delete any GoDaddy parking/forwarding records if present
- ☐ Add or edit these records:

| Type  | Name | Value                | TTL    |
|-------|------|----------------------|--------|
| A     | @    | 76.76.21.21          | 1 Hour |
| CNAME | www  | cname.vercel-dns.com | 1 Hour |

- ☐ Save changes
- ☐ Go back to the Vercel Domains page — wait for the green checkmark (5–30 min, sometimes up to 48 hours)
- ☐ Vercel provisions a free SSL certificate automatically

### Step 5: Set up aegisimaging.org redirect (optional)

- ☐ **Option A — Via Vercel:** Add `aegisimaging.org` as a domain in the same Vercel project. Vercel will redirect it to the primary domain.
- ☐ **Option B — Via GoDaddy:** On the `.org` domain, set up a domain forward: GoDaddy → My Products → aegisimaging.org → Manage → Forwarding → Forward to `https://aegisimaging.ai`

### Step 6: Verify everything works

- ☐ Visit `https://aegisimaging.ai` — page loads with SSL
- ☐ Visit `https://www.aegisimaging.ai` — redirects to apex
- ☐ Scroll through all sections — animations trigger
- ☐ Test mobile layout (resize browser or use phone)
- ☐ Submit the contact form — check inbox at `contact@aegisimaging.ai`
- ☐ Visit `https://aegisimaging.org` — redirects to `.ai` (if configured)

### How the contact form works

The contact form ( `POST /api/contact` ) has two independent delivery paths:

| Path | Backend | When |
|------|---------|------|
|------|---------|------|

|                                   |                                                           |                                     |
|-----------------------------------|-----------------------------------------------------------|-------------------------------------|
| <b>Vercel serverless function</b> | frontend/landing/api/contact.ts — nodemailer + Brevo SMTP | Landing page on Vercel (standalone) |
| <b>Go API endpoint</b>            | api/handler/contact.go — existing SMTP config             | Landing page proxied to Go backend  |

Both accept `{ name, email, organization, role, message }` and return `{ sent: true }`. The Vercel function is self-contained — it doesn't depend on the Go API. This means the landing page and contact form work even before the backend is deployed.

## 9. GitHub Repository

### Organization setup

Use GitHub Organizations to separate codebases by company.

- ☐ Create GitHub Organization: `aegis-imaging`
- ☒ Repo at `aegis-imaging/aegis` — transfer complete
- ☒ Local remote: `git@github.com:aegis-imaging/aegis.git`

### Repository setup

- ☐ Verify remote is set: `git remote -v`
- ☐ Push monorepo scaffold to `develop` branch
- ☐ Set up branch protection on `main` and `develop` (require PR reviews, prevent deletion)
  - **Requires GitHub Pro** (\$4/month) for private repos, or make the repo public
  - Go to Settings → Branches → Add branch protection rule
  - Branch name patterns: `main` and `develop`
  - Enable: "Require a pull request before merging", "Do not allow deletions"
- ☐ CI is configured: `.github/workflows/ci.yml` runs automatically on PRs to `develop` and `main`
  - Go build + vet, Python syntax check (7 services), TypeScript type check (5 apps), Docker build (8 images)
- ☐ Verify CI passes: open a test PR and check the Actions tab
- ☐ Run `make lint` locally to validate before pushing

## 10. Future — Before Proposing to Work

- ☐ Have a working end-to-end demo: upload → anonymize → view in Weasis
- ☐ Prepare a 5-minute screen recording of the demo flow
- ☐ Draft a one-page proposal covering: problem, solution, differentiation, cost estimate
- ☐ Identify potential pilot users / departments at your institution
- ☐ Research your company's internal startup / innovation program requirements

---

Generated 2026-02-18. Updated 2026-02-23. See *AEGIS\_Architecture.md* for the full system design. DIMSE receiver Compute Engine VM, Cloud Build CI/CD triggers, IAM hardening, DICOM conformance statement, and alert runbooks reflected through 2026-02-23.